Техническая документация Asciidoc и PlantUml

Обновлено 11 сентября 2025, 17:16 Иванов Игорь

Что это? Для кого?

Техническая документация на сервисы(микросервисы) Backend создаваемая Системным аналитиком для backend-разработчика.

Документация описывает:

- входные/выходные контракты методов API сервиса
- алгоритм выполнения методов API (диаграммы активности)
- таблицы маппинга
- структуру БД
- коды HTTP-ошибок с примерами
- алгоритм выполнения и контракты исходящих запросов на внешние(DonateHub) и внутренние ресурсы(h2h)

Все это пишется в форматах Asciicode(.adoc) и Plantuml(.puml)

Почему Asciidoc а не Markdown?

Ascii Doc обладает обширным функционалом, который необходим для качественной документации. К сожалению, Markdown подходит только для описания простой документации, уровня README.

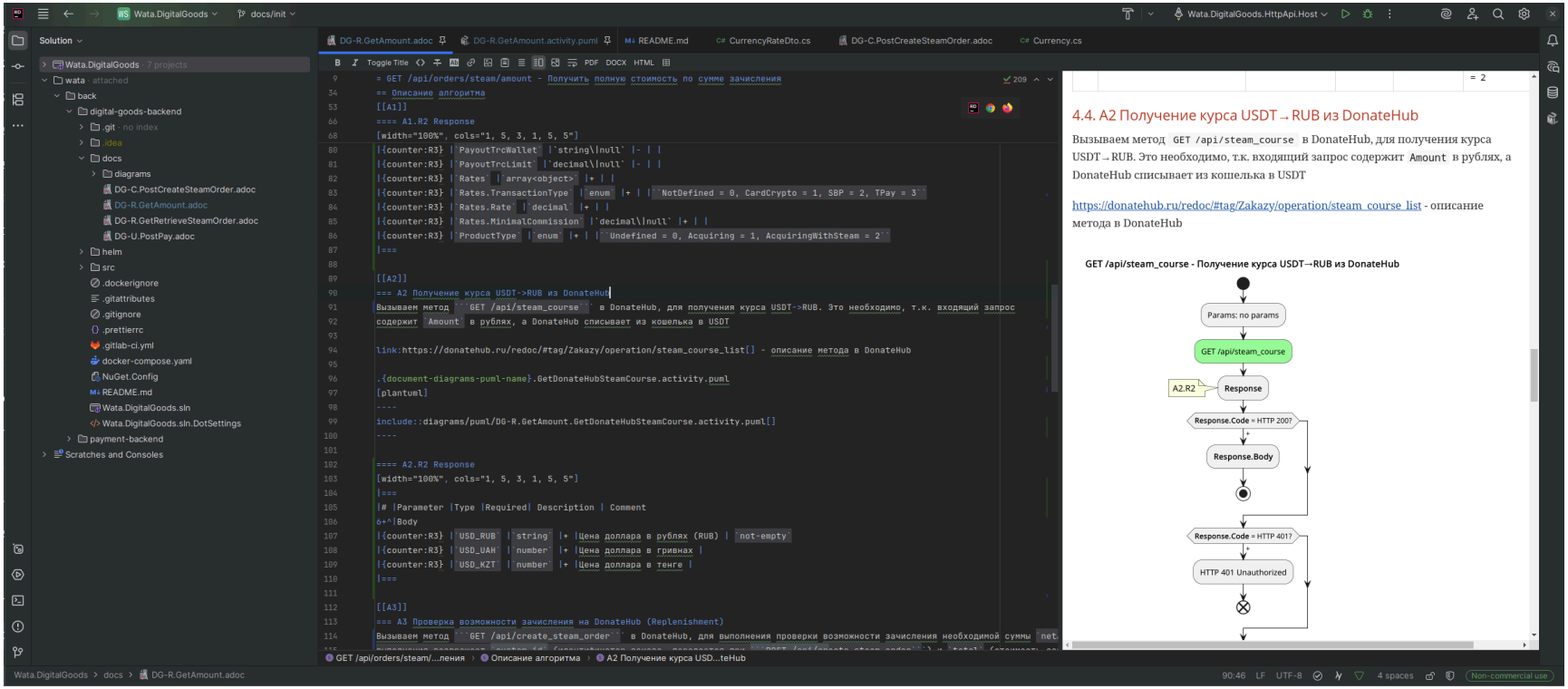
PlantUml - выдает классные диаграммы в строгом классическом оформлении. Стандарт де-факто

Как читать форматы adoc и puml?

Просмотр в JetBrains Rider

JetBrains Rider умеет рендерить файлы adoc и puml в режиме реального времени. Для этого необходимо установить плагины:

1. AsciiDoc - IntelliJ IDEs P... @JetBrains Marketplace
2. @JetBrains Marketplace plantuml4idea - IntelliJ I...



Чтобы использовать светлую тему для предпросмотра asciidoc (даже если тема в Rider темная), нужно задать в настройках:

1. Откройте **Settings(Preferences)** в Rider
2. Перейдите в раздел: **Languages & Frameworks** → **AsciiDoc**
3. В пункте **Preview Theme** выберите вариант "Asciidoc Light"

ПРИМЕЧАНИЕ: Системные аналитки используют светлую тему для создания документации. В темной теме некоторые элементы (например, подсветка вставок кода в абзаце) может выглядеть неэстетично.

Просмотр в Gitlab на Web

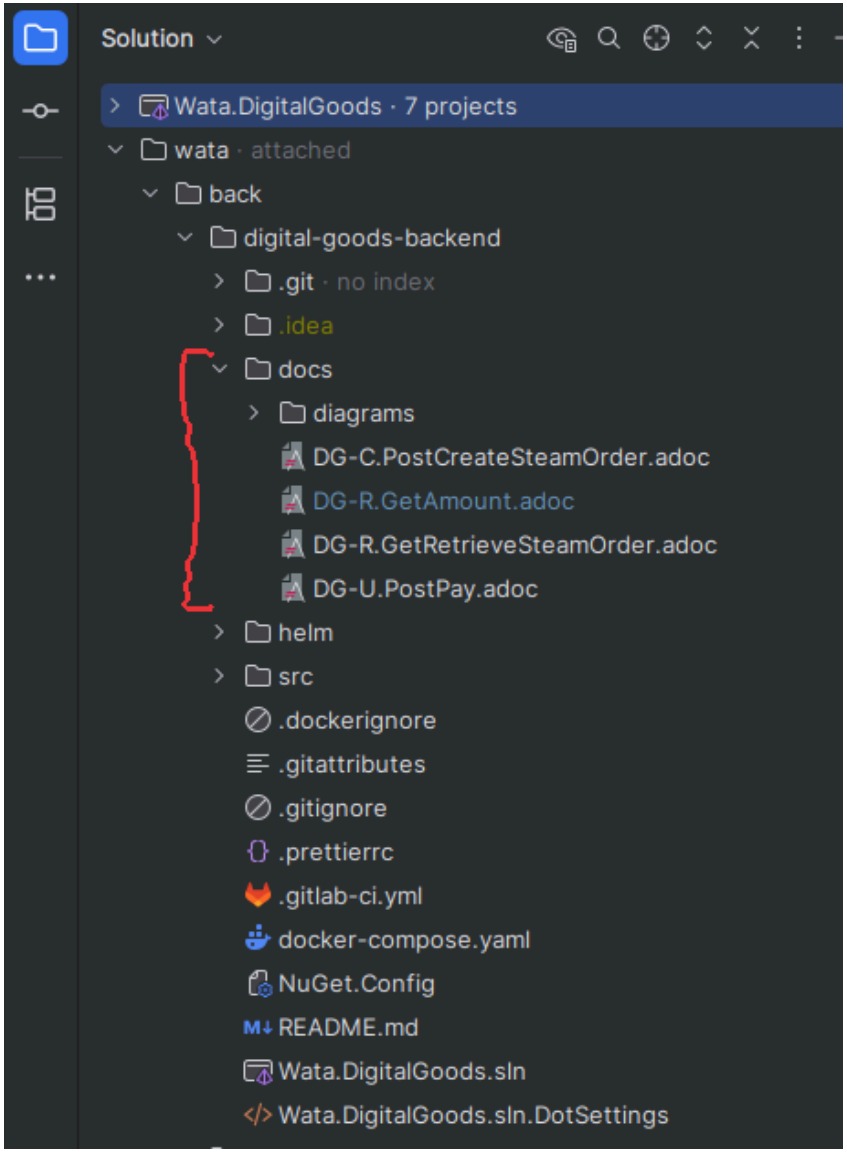
adoc и puml файлы автоматически рендерятся на Gitlab, по аналогии с md файлами. Доп настройки на браузере не требуются

ВНИМАНИЕ: На данный момент 2025-08-13 puml диаграммы на нашем self-hosted Gitlab временно не рендерятся, скоро будут доступны

Где находится документация?

Файлы adoc с диаграммами puml находятся в исходном коде сервиса, в папке `/docs`. Системный аналитик самостоятельно заливает файлы документации в git (подробнее см. далее)

Например, для сервиса DigitalGoods:



Какой процесс интеграции документации в git?

Документирование уже существующего кода (as-is)

Продолжается пока документация не догонит код

1. Системный аналитик
    1. подтягивает(`git pull`) исходный код из Gitlab из ветки `main`
    2. создает новую ветку из `main` , Например, `docs/TASK-2542`
    3. читает код, анализирует документацию(h2h Api, DonateHub Api, сиквенсы и др)
    4. вносит изменения в папке `/docs` . Файлы в формате adoc и puml
    5. делает коммит и пушит ветку документации в Gitlab (в удаленном репозитории появляется новая ветка с изменениями доки `docs/TASK-2542`)
    6. создает мерж-реквест: `docs/TASK-2542` -> `main`
2. Разработчик
    1. делает ревью мерж-реквеста
        1. Если OK, то заливает доки в `main`
        2. Если не OK, то оставляет замечания/вопросы для аналитика в мерж-реквесте
            1. Аналитик вносит правки

Документирование новых задач (to-be)

Когда документация опережает код - это необходимое условие для варианта to-be

ПРИМЕЧАНИЕ: Это упрощенный пример, в реальности процесс может быть более сложным

1. Продукт и бизнес анализ:
    1. Создают ФЗ
2. Системный аналитик:
    1. Изучает ФЗ
    2. <повторяет 1-пункт из варианта as-is>
    3. Проводится ревью документации другим аналитиком или разработчиком. Ветка **НЕ заливается** в `main`
3. Разработчик
    1. Знакомится с докой на новую задачу по измененям документации (изменение тех-документации идет как часть описания задачи)
    2. Создает ветку на dev-задачу. Например `feature/DEV-9871`

3. Подтягивает изменение документации из ветки `docs/TASK-2542` на свою dev-ветку `feature/DEV-9871`.
4. Реализует dev-задачу
5. <стандартный флоу разработки>
6. таким образом, изменения кода заливаются в ветки (dev, prod и т.п.) вместе с изменениями документации
7. удаляет ветку с документацией `docs/TASK-2542`

В каких сервисах это будет практиковаться?

В данный момент 2025-08-13, завершается документирование (**as-is**) сервиса Digital Goods. Документация уже доступна в ветке `main`

Ссылка на постоянную рабочую ветку `docs/init`: "https://wtcode.gitlab.yandexcloud.net/wata/back/digital-goods-backend/-/tree/docs/init/docs".
Нужно для просмотра задач находящихся в анализе/разработке